

4.6.5

Mahesh Chollangi - EE25BTECH11017

September, 2025

Question

Find the equation of the line passing through point $\mathbf{A} = (2, 1, -1)$ and parallel to the line.

$$\mathbf{r} = (\mathbf{i} + \mathbf{j}) + \lambda(2\mathbf{i} - \mathbf{j} + \mathbf{k}).$$

Also, find the distance between these two lines.

Direction Vector

The direction vector ***m*** of the given line is

$$\mathbf{m} = \begin{pmatrix} 2 \\ -1 \\ 1 \end{pmatrix} \quad (1)$$

The point through which the required line passes is

$$\mathbf{A} = \begin{pmatrix} 2 \\ 1 \\ -1 \end{pmatrix} \quad (2)$$

Equation of the Lines

The equation of the required line is:

$$\mathbf{r}_1 = \mathbf{A} + \lambda \mathbf{m} = \begin{pmatrix} 2 \\ 1 \\ -1 \end{pmatrix} + \lambda \begin{pmatrix} 2 \\ -1 \\ 1 \end{pmatrix} \quad (3)$$

The equation of the given line is:

$$\mathbf{r}_2 = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} + \mu \begin{pmatrix} 2 \\ -1 \\ 1 \end{pmatrix} \quad (4)$$

Distance Between the Lines

The distance d between the two parallel lines with the same direction vector $\mathbf{m}$ is:

$$d = \frac{\|(\mathbf{A} - \mathbf{B}) \times \mathbf{m}\|}{\|\mathbf{m}\|} \quad (5)$$

where

$$\mathbf{B} = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}.$$

Compute Vector Difference

Calculate the vector difference:

$$\mathbf{A} - \mathbf{B} = \begin{pmatrix} 2 \\ 1 \\ -1 \end{pmatrix} - \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ -1 \end{pmatrix} \quad (6)$$

Cross Product Calculation

Calculate the cross product $(\mathbf{A} - \mathbf{B}) \times \mathbf{m}$:

$$\begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ 1 & 0 & -1 \\ 2 & -1 & 1 \end{vmatrix}$$

Expanding:

$$= \mathbf{i}(0 \times 1 - (-1) \times (-1)) - \mathbf{j}(1 \times 1 - (-1) \times 2) + \mathbf{k}(1 \times (-1) - 0 \times 2) \quad (7)$$

$$= \mathbf{i}(0 - 1) - \mathbf{j}(1 + 2) + \mathbf{k}(-1 - 0) \quad (8)$$

$$= -\mathbf{i} - 3\mathbf{j} - \mathbf{k} \quad (9)$$

Magnitude Calculations

Magnitude of the cross product:

$$\|(\mathbf{A} - \mathbf{B}) \times \mathbf{m}\| = \sqrt{(-1)^2 + (-3)^2 + (-1)^2} = \sqrt{11} \quad (10)$$

Magnitude of the direction vector:

$$\|\mathbf{m}\| = \sqrt{2^2 + (-1)^2 + 1^2} = \sqrt{6} \quad (11)$$

Final Distance and Conclusion

Hence, the distance between the two lines is:

$$d = \frac{\sqrt{11}}{\sqrt{6}} \quad (12)$$

and the required line equation is:

$$\mathbf{r}_1 = \begin{pmatrix} 2 \\ 1 \\ -1 \end{pmatrix} + \lambda \begin{pmatrix} 2 \\ -1 \\ 1 \end{pmatrix} \quad (13)$$

C Code

```
#include <math.h>

// Given data
// Point on first line: P1 = (1,1,0)
// Direction vector of first line: d1 = (2,-1,1)
// Point on second line: P2 = (2,1,-1)
// Direction vector of second line (same as d1 since
// parallel)

double P1[3] = {1, 1, 0};
double d1[3] = {2, -1, 1};
double P2[3] = {2, 1, -1};
double d2[3] = {2, -1, 1};

double cross[3];

double cross_norm;

double distance;
```

C Code (continued)

```
// Function to compute cross product
void cross_product(double a[3], double b[3], double
    result[3]) {
    result[0] = a[1]*b[2] - a[2]*b[1];
    result[1] = a[2]*b[0] - a[0]*b[2];
    result[2] = a[0]*b[1] - a[1]*b[0];
}

// Function to compute dot product
double dot_product(double a[3], double b[3]) {
    return a[0]*b[0] + a[1]*b[1] + a[2]*b[2];
}

// Function to compute norm
double norm(double a[3]) {
    return sqrt(dot_product(a, a));
}
```

C Code (continued)

```
void compute_distance() {
    double P2_minus_P1[3] = {P2[0]-P1[0], P2[1]-P1[1],
        P2[2]-P1[2]};

    // Compute cross product of direction vectors (
    // should be zero or near zero since parallel)
    cross_product(d1, d2, cross);
    cross_norm = norm(cross);

    if (cross_norm < 1e-8) {
        // Lines are parallel
        // distance = |(P2 - P1) x d1| / |d1|
        double temp[3];
        cross_product(P2_minus_P1, d1, temp);
        distance = norm(temp) / norm(d1);
    } else {
        // Lines are skew
        // distance = |(P2 - P1) . (d1 x d2)| / |d1 x
        d2|
```

C Code (continued)

```
1         distance = fabs(dot_product(P2_minus_P1, cross
2             )) / cross_norm;
3     }
4 }
5
6 // Getter function for distance
7 double get_distance() {
8     return distance;
9 }
10
11 // Getter functions for points and direction vectors
12
13 void get_P1(double* arr) {
14     for (int i=0; i<3; i++)
15         arr[i] = P1[i];
16 }
```

C Code (continued)

```
void get_P2(double* arr) {  
    for (int i=0; i<3; i++)  
        arr[i] = P2[i];  
}  
  
void get_direction(double* arr) {  
    for (int i=0; i<3; i++)  
        arr[i] = d1[i];  
}
```

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

# Given line:  $r = (i + j) + \lambda(2i - j + k)$ 
point1 = np.array([1, 1, 0]) # position vector  $(i + j)$ 
direction1 = np.array([2, -1, 1]) # direction vector

# Required line passing through  $(2, 1, -1)$  and
# parallel to given line
point2 = np.array([2, 1, -1])
direction2 = direction1.copy()

# Function to parameterize points on line
def line_points(point, direction, t_range):
    return np.array([point + t * direction for t in
                     t_range])
```

Python Code (continued)

```
# Generate points on both lines
lambda_range = np.linspace(-5, 5, 100)
line1_points = line_points(point1, direction1,
                             lambda_range)
line2_points = line_points(point2, direction2,
                             lambda_range)

# Compute distance between two skew lines using
  formula:
# distance = |(P2 - P1) . (d1 x d2)| / |d1 x d2|
P2_minus_P1 = point2 - point1
cross_d1_d2 = np.cross(direction1, direction2)
cross_norm = np.linalg.norm(cross_d1_d2)

if cross_norm == 0:
    # Lines are parallel
    distance = np.linalg.norm(np.cross(P2_minus_P1,
                                         direction1)) / np.linalg.norm(direction1)
```


Python Code (continued)

```
else:
    distance = abs(np.dot(P2_minus_P1, cross_d1_d2)) /
        cross_norm

print(f"Distance between the two lines = {distance:.4f}
      ")

# Plotting
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.plot(line1_points[:,0], line1_points[:,1],
        line1_points[:,2], label='Given line')
ax.plot(line2_points[:,0], line2_points[:,1],
        line2_points[:,2], label='Required line (parallel)')
)
```

Python Code (continued)

```
ax.scatter(*point1, color='blue')
ax.scatter(*point2, color='green')

ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title(f'Distance between lines = {distance:.4f}')
ax.legend()
plt.show()
```

Distance between lines = -2003689232.0000

